



1

Chapitre 4: Javascript



Dorra DHAOU

Maître assistante en Informatique

Docteur et Ingénieur en Informatique

dorra.dhaou@fst.utm.tn / dorradhaou@gmail.com

AU. 2025 - 2026

1

Introduction

2

- Javascript est un langage de scripts, orienté objet et interprété.
- Il peut être intégré dans le code HTML des pages Web.
- Il permet d'améliorer leurs présentations et interactions avec les visiteurs.
- Le code javascript est interprété et exécuté côté client par le navigateur Web.
 - ▣ Le temps de réponse est assez réduit

2

Plan

3

- Exemple du javascript
- Intégration du javascript
- Entrées/sorties
- ...

3

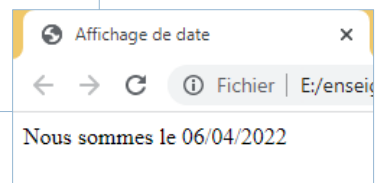
Exemple du javascript

4

Affichage de date

- Avec HTML, on ne peut créer que des pages Web statiques dont le contenu ne peut pas changer.

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
  </head>
  <body>
    Nous sommes le 06/04/2022
  </body>
</html>
```



4

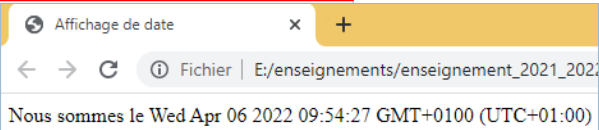
Exemple du javascript

5

Affichage de date

- Avec HTML et javascript, on peut créer des pages Web dynamiques dont le contenu peut changer.

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
  </head>
  <body>
    <script type = "text/javascript">
      var date = new Date();
      document.write("Nous sommes le " + date);
    </script>
  </body>
</html>
```



5

6

Intégration du javascript

6

Intégration du javascript

7

- Le code javascript peut être intégré dans un document HTML avec quatre méthodes dans:
 - ▣ L'entête du document (balise **<head>**) en utilisant la balise **<script>**
 - ▣ Le corps du document (balise **<body>**) en utilisant la balise **<script>**
 - ▣ Un fichier externe « .js » en utilisant la balise **<script>** dans l'entête ou le corps du document
 - ▣ Un évènement d'un élément (clic, soumission, etc.)

7

Intégration du javascript

8

Méthode 1 - entête HTML

- La balise **<script>** permet de définir la fonction javascript dans l'entête du document HTML.
 - ▣ L'attribut **type** permet de définir le type du contenu (text/javascript par défaut)
- Exemple: affichage de date (définition)

```

<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
    <script type = "text/javascript">
      function afficherDate(){
        var date = new Date();
        return "Nous sommes le " + date;}
    </script>
  </head>

```

8

Intégration du javascript

9

Méthode 1 - entête HTML

- La balise `<script>` permet d'appeler la fonction javascript déclarée dans l'entête (balise `<head>`) dans le corps du document HTML (balise `<body>`).
- Exemple: affichage de date (appel)

```
<body>
  <script>
    document.write(afficherDate());
  </script>
</body>
</html>
```

9

Intégration du javascript

10

Méthode 1 - entête HTML

- Exemple: affichage de date (définition + appel)

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
    <script type = "text/javascript">
      function afficherDate(){
        var date = new Date();
        return "Nous sommes le " + date;}
    </script>
  </head>
  <body>
    <script>
      document.write(afficherDate());
    </script>
  </body>
</html>
```

Affichage de date x +

← → ↻ ⓘ Fichier | E:/enseignements/enseignement_2021_2022

Nous sommes le Wed Apr 06 2022 10:26:25 GMT+0100 (UTC+01:00)

10

Intégration du javascript

11

Méthode 2 - corps HTML

- La balise **<script>** permet d'appeler une fonction ou exécuter une commande javascript dans le corps du document HTML.
- Exemple: affichage de date

```

<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
  </head>
  <body>
    <script type = "text/javascript">
      var date = new Date();
      document.write("Nous sommes le " + date);
    </script>
  </body>
</html>

```

11

Intégration du javascript

12

Méthode 3 – fichier externe

- La balise **<script>** permet d'intégrer du code javascript écrit dans un fichier externe.
- Elle peut être insérée dans:
 - ▣ L'entête du document HTML pour une exécution **différée**
 - ▣ Le corps du document HTML pour une exécution **directe**
- L'attribut **src** permet de spécifier le fichier externe à intégrer.

12

Intégration du javascript

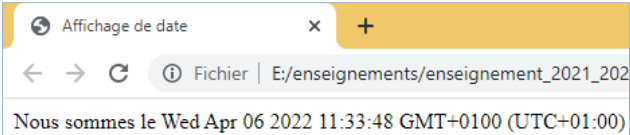
13

Méthode 3 – fichier externe

- Exemple: affichage de date (exécution différée)

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
    <script type = "text/javascript" src = "script.js"> </script>
  </head>
  <body>
    <script>
      document.write(afficherDate());
    </script>
  </body>
</html>
```

```
function afficherDate() {
  var date = new Date();
  return "Nous sommes le " + date;
}
```



13

Intégration du javascript

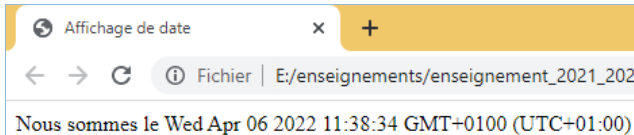
14

Méthode 3 – fichier externe

- Exemple: affichage de date (exécution directe)

```
<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
  </head>
  <body>
    <script type = "text/javascript" src = "script2.js"> </script>
  </body>
</html>
```

```
var date = new Date();
document.write("Nous sommes le " + date);
```



14

Intégration du javascript

15

Méthode 4 – évènement d'un élément HTML

- Il existe plusieurs évènements relatifs aux éléments HTML (clic, soumission, réinitialisation, etc.).
- Dans une balise HTML, un évènement est identifié par un attribut (**onclick**, **onsubmit**, **onreset**, etc.).
- La valeur de chaque attribut doit comprendre le code javascript à exécuter une fois l'évènement réalisé.

15

Intégration du javascript

16

Méthode 4 – évènement d'un élément HTML

- Exemple: affichage de date

The screenshot shows an HTML editor on the left and a web browser on the right. The editor contains the following code:

```

<!doctype html>
<html>
  <head>
    <title> Affichage de date </title>
    <script type = "text/javascript" src = "script3.js">
    </script>
  </head>
  <body>
    <input type = "button" value = "Afficher la date" onclick = "afficherDate()"/>
  </body>
</html>

```

The browser window shows the title "Affichage de date" and a button labeled "Afficher la date". The console output displays the current date and time: "Nous sommes le Wed Apr 06 2022 11:59:32 GMT+0100 (UTC+01:00)".

16

17

Entrées/sorties

17

Entrées/sorties

18

- En javascript, trois fonctions peuvent être utilisées pour afficher une boîte de dialogue aux visiteurs:
 - ▣ **alert()**
 - Afficher un message aux visiteurs avec un bouton « **OK** »
 - ▣ **confirm()**
 - Fournir un choix aux visiteurs avec deux boutons « **OK** » et « **Annuler** »
 - ▣ **prompt()**
 - Permettre aux visiteurs de saisir un message, une valeur, etc. avec deux boutons « **OK** » et « **Annuler** »

18

Entrées/sorties

19

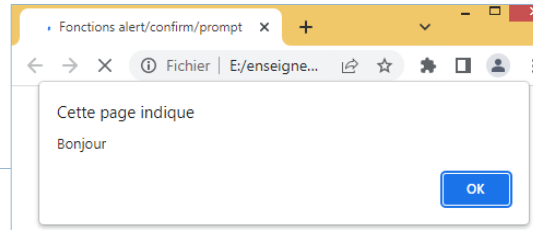
Fonction alert()

- Le message à afficher aux visiteurs doit être passé comme paramètre de la fonction **alert()**.

- Exemple:

```
alert("Bonjour");
```

```
<!doctype html>
<html>
  <head>
    <title> Fonctions alert/confirm/prompt </title>
  </head>
  <body>
    <script type = "text/javascript" src = "script_in_out.js"> </script>
  </body>
</html>
```



19

Entrées/sorties

20

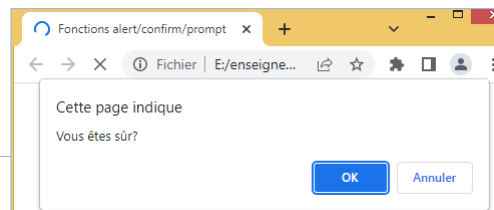
Fonction confirm()

- Le message à afficher aux visiteurs doit être passé comme paramètre de la fonction **confirm()**.

- Exemple:

```
confirm("Vous êtes sûr?");
```

```
<!doctype html>
<html>
  <head>
    <title> Fonctions alert/confirm/prompt </title>
  </head>
  <body>
    <script type = "text/javascript" src = "script_in_out.js"> </script>
  </body>
</html>
```



20

Entrées/sorties

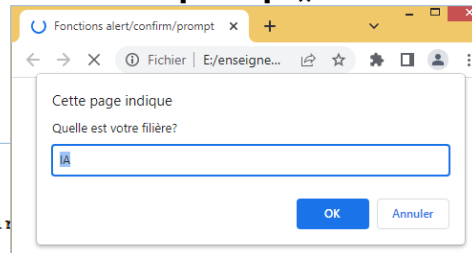
21

Fonction prompt()

- Le message à afficher aux visiteurs et le texte par défaut du champ de saisie doivent être passés comme paramètres de la fonction **prompt()**.

- Exemple:

```
<!doctype html>
<html>
  <head>
    <title> Fonctions alert/confirm
  </head>
  <body>
    <script type = "text/javascript" src = "script_in_out.js"> </script>
    <script>
      prompt("Quelle est votre filière?", "IA");
    </script>
  </body>
</html>
```



21

Entrées/sorties

22

Exercice 1

- Écrire un script javascript permettant de demander le nom et l'âge du visiteur et les afficher dans une boîte de dialogue séparés par une virgule.

```
var nom = prompt("Donner votre nom:");
var age = prompt("Donner votre âge:");
var msg = nom + ',' + age;
alert(msg);

<!doctype html>
<html>
  <head>
    <title> Exercice 1 </title>
    <meta charset = "UTF-8"/>
  </head>
  <body>
    <script type = "text/javascript" src = "script_exercice1.js"> </script>
  </body>
</html>
```

22

Entrées/sorties

23

Exercice 2

- Écrire un script javascript permettant de demander la saisie de deux entiers et d'afficher la somme.

```

var a = prompt("Donner un entier:");
var b = prompt("Donner un entier:");
var res = parseInt(a)+parseInt(b);
alert("La somme = "+res);

```

```

<!doctype html>
<html>
  <head>
    <title> Exercice 2 </title>
    <meta charset = "UTF-8"/>
  </head>
  <body>
    <script type = "text/javascript" src = "script_exercice2.js"> </script>
  </body>
</html>

```

23

La Console JavaScript

24

- La console est un outil intégré au navigateur (accessible via la touche **F12** ou **Clic droit > Inspecter**).
- Elle est indispensable pour le développeur car elle permet de :

A. Déboguer (Contrôler le contenu)

- Elle permet d'afficher la valeur des variables à un instant précis pour vérifier si le programme récupère les bonnes données.
- On utilise la commande : `console.log(maVariable);`

B. Tracer (Suivre l'exécution)

- Elle permet de vérifier quels blocs de code (conditions, boucles, fonctions) sont réellement visités par le programme.
- Cela permet de comprendre le cheminement logique du script et de repérer où l'exécution s'arrête.

24

La Console JavaScript

25

Pourquoi utiliser la console ?

- **Gain de temps** : On évite d'utiliser trop d'`alert()` qui bloquent l'interaction utilisateur.
- **Précision** : Elle affiche les erreurs de syntaxe précises et la ligne exacte où le code a échoué.
- **Discrétion** : Les messages envoyés à la console ne sont pas visibles par le visiteur du site, seulement par le développeur.

25

La Console JavaScript

26

Exécution hors navigateur (NodeJS)

- Il est possible d'exécuter du code JavaScript sans navigateur en utilisant **NodeJS**.
- **NodeJS** est un environnement d'exécution qui permet d'utiliser JavaScript côté serveur ou sur votre propre machine.

Comment tester son programme ?

- Ouvrir une console (Invite de commandes, PowerShell ou Cmdr).
- Se placer dans le dossier contenant votre fichier.
- Lancer la commande : `node nomFichier.js`

26

La Console JavaScript

27

□ **Exemple:** fichier file.js

```
function maFonction() {  
  console.log("Hello World !");  
}  
maFonction();
```

→ Lancer la commande `node file.js`

Résultat affiché dans la console : Hello World !

□ **Il est aussi possible de définir un raccourci de** `console.log`

```
var c1 = console.log;  
c1("Hello World !");
```

27

28

Concepts de base

28

Variables

29

- En JavaScript, on peut déclarer des variables avec trois mots-clés :
 - ▣ **var** : Ancienne syntaxe pour déclarer une variable (portée de fonction).
 - ▣ **let** : Syntaxe moderne pour déclarer une variable dont la valeur peut changer (portée de bloc).
 - ▣ **const** : Utilisé pour déclarer une constante dont la valeur ne peut pas être modifiée après affectation.

Exemples :

```
var nom = "Ali";
let age = 25;
const pays = "Tunisie";
```

29

Portée des variables déclarées

30

- Une variable déclarée avec le mot-clé **var** peut avoir une **portée globale**.

```
{
  var x = 1;
}
console.log(x); // affiche : 1
```

- Même si la variable **X** est déclarée à l'intérieur d'un bloc **{ }**, elle reste **accessible à l'extérieur**. (**X** reste accessible **en dehors du bloc { }**)

Remarque

- En JavaScript, une variable déclarée dans un bloc **{ ... }** (comme dans **if**, **for**, etc.) avec **var** a une **portée globale ou de fonction**, et non une portée limitée au bloc.

30

Portée des variables déclarées

31

- Une variable déclarée avec le mot-clé **let** possède une **portée limitée au bloc { ... }** dans lequel elle est déclarée.

```
{
  let x = 1;
}
console.log(x); // affiche : erreur : x is not defined
```

- Dans cet exemple, la variable **x** n'est accessible qu'à l'intérieur du bloc.

Remarque

- En JavaScript, Une variable déclarée avec **let** dans un bloc (if, for, {...}) n'est pas accessible en dehors de ce bloc.

31

Portée des variables déclarées

32

- Une variable déclarée avec **const** possède également une **portée limitée au bloc { ... }**.

```
{
  const y = 5;
}
console.log(y); // affiche : erreur : y is not defined
```

- Dans cet exemple, la variable **y** n'est accessible qu'à l'intérieur du bloc.

Remarque

- En JavaScript, **const** empêche la modification de la valeur après son initialisation/affectation.
- Comme **let**, sa portée est limitée au bloc.

32

Permuter le contenu de deux variables

33

- En JavaScript, on peut **échanger les valeurs de deux variables** grâce à la **déstructuration** :

```
let a = 2;
let b = 0;
[a, b] = [b, a];
console.log(a); // 0
console.log(b); // 2
```

- La syntaxe `[a, b] = [b, a]` permet de **permuter les valeurs sans utiliser de variable temporaire**.

33

Types de données

34

- JavaScript utilise plusieurs types de données fondamentaux :
 - ▣ **Number** : Pour les nombres (entiers ou décimaux).
 - ▣ **String** : Pour les chaînes de caractères (entre " " ou ' ').
 - ▣ **Boolean** : Pour les valeurs logiques (true ou false).
 - ▣ **Object** : Pour stocker des collections de données.
 - ▣ **Array** : Pour stocker des listes d'éléments.

Exemples :

```
let score = 95;           // Number
let message = "Bravo";    // String
let estValide = true;     // Boolean
```

34

Opérateur typeof

35

- **Récupérer le type de la valeur stockée dans une variable**
- En JavaScript, l'opérateur **typeof** permet de connaître le **type d'une valeur ou d'une variable**.


```

console.log(typeof 5);           // number
console.log(typeof true);        // boolean
console.log(typeof 5.2);         // number
console.log(typeof "bonjour");   // string
console.log(typeof 'c');         // string
var x;
console.log(typeof x);           // undefined
console.log(typeof {nom:"Ben foulén", prenom:"Foulén"}); // object
console.log(typeof new Date()); // object
console.log(typeof Date);        // function
console.log(typeof [1,2,3,4]);   // object
console.log(typeof null);        // object

```
- ➔ Pour les **objets prédéfinis en JavaScript**, **typeof** peut retourner deux types principaux : **object** (si c'est une **instance d'objet**) ou **function** (**constructeur ou fonction normale**)

35

Opérateurs arithmétiques

36

- L'opérateur **+** possède deux fonctions en JavaScript :
 - **L'addition numérique** : si les deux opérandes sont des nombres.
 - **La concaténation** : si l'un des opérandes est une **chaîne de caractères** (string), JavaScript transforme l'autre en chaîne et les "colle" ensemble.

Remarques :

- **La priorité de gauche à droite.**
- **Le type "String" gagne toujours** : Dès qu'une chaîne apparaît dans une suite d'additions, le reste du résultat devient souvent une chaîne.
- **NaN (Not a Number)** : Apparaît lorsqu'on essaie de faire une opération mathématique avec une variable déclarée mais non initialisée.

36

Opérateurs arithmétiques

37

```

var x = 1;
y = 3;
z = '8';
t = "2";

u = "bonjour";
v = "3bonjour";
var m;
n = 2.5;

console.log(x + y); // 1 + 3 = 4
console.log(x + z); // 1 + '8' → '18' (concaténation car '8' est une chaîne)
console.log(x + t); // 1 + "2" → "12"
console.log(x + y + z); // (1 + 3) + '8' → 4 + '8' → "48"
console.log(x + u); // 1 + "bonjour" → "1 bonjour"
console.log(x + v); // 1 + "3bonjour" → "13bonjour"
console.log(u + v); // "bonjour" + "3bonjour" → "bonjour3bonjour"
console.log(x + m); // 1 + undefined → NaN (Not a Number)
console.log(x + n); // 1 + 2.5 = 3.5

```

37

Opérateurs arithmétiques

38

□ En plus de l'addition (+), JavaScript utilise les opérateurs suivants :

- * : Multiplication
- - : Soustraction
- / : Division
- % : Modulo (reste de la division entière)
- ** : Exposant (ex: 2**3 donne 8)

Quelques raccourcis d'écriture (Incrémentations) :

- i++; équivaut à i = i + 1;
- i--; équivaut à i = i - 1;
- i += 2; équivaut à i = i + 2;
- i *= 3; équivaut à i = i * 3;

38

L'opérateur unaire + et -

39

- L'opérateur **unaire +** permet de **convertir une valeur en nombre**.

```
console.log(typeof (+ "3")); // affiche : number
console.log(+true);         // affiche : 1
```

- "3" (chaîne de caractères) devient le nombre 3.
- true devient 1, et false deviendrait 0.

- L'opérateur **unaire -** sert à **négaliver une valeur** ou à la **convertir en nombre négatif**.

```
console.log(typeof (- "3")); // affiche : number
console.log(- "3");         // affiche : 3
console.log(-true);         // affiche : -1
```

- "-3" → -3, et ensuite -(-3) → 3.
- true devient 1, donc -true → -1.

39

La fonction eval()

40

- La fonction **eval()** permet d'évaluer une **expression arithmétique écrite sous forme de chaîne de caractères** :

```
var str = "2 + 5 * 3";
console.log(eval(str)); // affiche : 17
```

Attention :

- si l'expression est mal formulée, **eval()** génère une **EvalError**.
- **eval()** exécute tout code JavaScript contenu dans la chaîne, donc son utilisation doit être prudente.

40

Déclaration des chaînes de caractères

41

- En JavaScript, une chaîne de caractères peut être déclarée de **deux façons différentes** :

```
var str1 = new String("bonjour");
console.log(typeof(str1)); // affiche : object

var str2 = "bonjour";
console.log(typeof(str2)); // affiche : string
```

- Avec **new String()** : on crée un objet String, donc typeof retourne object.
- Avec **"..."** : on crée une chaîne de caractères simple, donc typeof retourne string.

41

Méthodes utiles pour les chaînes de caractères

42

- En JavaScript, les chaînes de caractères disposent de plusieurs **méthodes pratiques** :

Méthode	Description
length	Retourne la longueur de la chaîne
toUpperCase()	Convertit la chaîne en majuscules
toLowerCase()	Convertit la chaîne en minuscules
trim()	Supprime les espaces au début et à la fin
substr(i, j)	Extrait une sous-chaîne à partir de l'indice i sur j caractères
indexOf(sub)	Retourne la position de la sous-chaîne sub dans la chaîne, ou -1 si elle n'existe pas
...	D'autres méthodes existent (ex : slice(), replace(), includes(), ...)

42

Accéder à un caractère dans une chaîne

43

□ Il existe **plusieurs façons** d'accéder à un caractère à l'indice *i* :

1. **Via l'indice directement** : `console.log(str[i]);`
2. **En extrayant une sous-chaîne** :
`console.log(str.substr(i, 1));` // extrait 1 caractère à partir de l'indice *i*
3. **Avec la méthode `charAt()`** : `console.log(str.charAt(i));`

Remarque

- Si vous appelez **une méthode sur une variable undefined**, JavaScript génère une **erreur `TypeError`**.

```
var str;
console.log(str.toUpperCase()); // TypeError : Cannot read property
                                'toUpperCase' of undefined
```

43

Structures de contrôle conditionnelles

44

Les conditions permettent de modifier le comportement du script selon des critères logiques.

- **La structure `if / else`** :
 - **`if`** : exécute un bloc si la condition est vraie.
 - **`else if`** : teste une nouvelle condition si la précédente est fausse.
 - **`else`** : s'exécute si aucune condition n'est remplie.

Exemple :

```
let score = 75;

if (score > 50) {
  alert("Résultat : Réussi");
} else {
  alert("Résultat : Échec");
}
```

44

Structures de contrôle conditionnelles

Opérateurs de comparaison

45

- Pour formuler les conditions, on utilise des opérateurs de comparaison et des opérateurs logiques :

- **Opérateurs logiques** : && : et

|| : ou

! : non

- **Opérateurs de comparaison** :

Opérateur	Description	Exemple
==	Égal à (valeur)	5 == "5" (Vrai)
===	Égal à (valeur ET type)	5 === "5" (Faux)
!= / !==	Différent de	a != b
>, <	Supérieur / Inférieur	score > 10
>=, <=	Supérieur ou égal...	age >= 18

45

Structures de contrôle conditionnelles

46

Les conditions permettent de modifier le comportement du script selon des critères logiques.

- **La structure switch** : idéale pour tester plusieurs valeurs d'une même variable.

C'est une alternative plus lisible au if/else if lorsqu'on doit comparer une même variable à plusieurs valeurs possibles.

- Mots-clés :

- **case** : définit une valeur à tester.
- **break** : arrête l'exécution et sort du bloc (très important).
- **default** : s'exécute si aucun cas ne correspond (optionnel).

46

Structures de contrôle conditionnelles

47

□ La structure switch :

Exemple :

La variable dans **switch** peut être :

- Un nombre
- Un caractère
- Une chaîne de caractères (contrairement aux java, C, C++ ...)

```
let jour = 2;
let nomJour;

switch (jour) {
  case 1:
    nomJour = "Lundi";
    break;
  case 2:
    nomJour = "Mardi";
    break;
  case 3:
    nomJour = "Mercredi";
    break;
  default:
    nomJour = "Jour inconnu";
}

alert("Aujourd'hui nous sommes : " + nomJour);
// Affiche "Mardi"
```

47

Structures de contrôle conditionnelles

48

□ Considérons l'exemple suivant :

```
var message = prompt('Saisir un message');
switch (message) {
  case "bonjour":
    alert('salut !');
    break;
  default:
    alert('quoi ?');
}
```

□ Version simplifiée avec l'opérateur ternaire

```
var message = prompt('Saisir un message');
message == "bonjour" ? alert("salut !") : alert('quoi ?');
```

➔ Le ternaire permet d'écrire un if/else en une seule ligne :

condition ? action_si_vrai : action_si_faux

- **switch** est pratique pour plusieurs cas possibles.
- Le ternaire est plus concis pour des tests simples à deux options.

48

Structures de contrôle itératives : Les boucles (loops)

49

- Les boucles servent à répéter un bloc d'instructions tant qu'une condition est vérifiée.
- **Types de boucles :**
 - ▣ **for** : utilisée quand on connaît à l'avance le nombre de répétitions.
 - ▣ **while** : Répéter tant qu'une condition est vraie (test au début).
 - ▣ **do...while** : Exécuter au moins une fois, puis répéter tant que la condition est vraie. (*répéter... jusqu'à* en algorithmique)

□ Exemples :

```
// Boucle FOR : affiche les nombres de 0 à 4
for (let i = 0; i < 5; i++) {
  console.log("Compteur : " + i);
}
```

```
// Boucle WHILE
let j = 0;
while (j < 3) {
  console.log("Passage n°" + j);
  j++;
}
```

49